

ГЛАВА 4

ВЫРАЖЕНИЯ

В этой главе...

4.1. Основы	188
4.2. Арифметические операторы	194
4.3. Логические операторы и операторы отношения	197
4.4. Операторы присвоения	201
4.5. Операторы инкремента и декремента	204
4.6. Операторы доступа к членам	207
4.7. Условный оператор	208
4.8. Побитовые операторы	210
4.9. Оператор <code>sizeof</code>	215
4.10. Оператор запятая	217
4.11. Преобразование типов	218
4.12. Таблица приоритетов операторов	227
Резюме	229
Термины	229

Язык C++ предоставляет богатый набор операторов, а также определяет их назначение и применение к операндам встроенного типа. Он позволяет также определять назначение большинства операторов, операндами которых являются объекты классов. Эта глава посвящена операторам, определенным в самом языке и применяемым к операндам встроенных типов. Будут описаны также некоторые из операторов, определенных библиотекой. Определение операторов для собственных типов рассматривается в главе 14.

Выражение (expression) состоит из одного или нескольких *операторов* (operator) и возвращает *результат* (result) вычисления. Самая простая форма выражения — это одиночный литерал или переменная. Более сложные выражения формируются из оператора и одного или нескольких *операндов* (operand).

4.1. Основы

Существует несколько фундаментальных концепций, определяющих то, как обрабатываются выражения. Начнем с краткого обсуждения концепций, относящихся к большинству (если не ко всем) выражений. В последующих разделах эти темы рассматриваются подробнее.



4.1.1. Фундаментальные концепции

Существуют *унарные операторы* (unary operator) и *парные операторы* (binary operator). Унарные операторы, такие как обращение к адресу (&) и обращение к значению (*), воздействуют на один операнд. Парные операторы, такие как равенство (==) и умножение (*), воздействуют на два операнда. Существует также (всего один) *тройственный оператор* (ternary operator), который использует три операнда, а также *оператор вызова функции* (function call), который получает неограниченное количество операндов.

Некоторые *символы* (symbol), например *, используются для обозначения как унарных (обращение к значению), так и парных (умножение) операторов. Представляет ли символ унарный оператор или парный, определяет контекст, в котором он используется. В использовании таких символов нет никакой взаимосвязи, поэтому их можно считать двумя разными символами.

Группировка операторов и операндов

Чтобы лучше понять порядок выполнения выражений с несколькими операторами, следует рассмотреть концепцию *приоритета* (precedence), *порядка* (associativity) и *порядка вычисления* (order of evaluation) операторов. Например, в следующем выражении используются сложение, умножение и деление:

```
5 + 10 * 20/2;
```

Операндами оператора * могли бы быть числа 10 и 20, либо 10 и 20/2, либо 15 и 20, либо 15 и 20/2. Понимание таких выражений и является темой следующего раздела.

Преобразование операндов

В ходе вычисления выражения операнды нередко преобразуются из одного типа в другой. Например, парные операторы обычно ожидают операндов одинакового типа. Но операторы применимы и к операндам с разными типами, если они допускают преобразование (см. раздел 2.1.2, стр. 66) в общий тип.

Хотя правила преобразования довольно сложны, по большей части они очевидны. Например, целое число можно преобразовать в число с плавающей запятой, и наоборот, но преобразовать тип указателя в число с плавающей точкой нельзя. Немного неочевидным может быть то, что операнды меньших

целочисленных типов (например, `bool`, `char`, `short` и т.д.) обычно *преобразуются* (promotion) в больший целочисленный тип, как правило `int`. Более подробная информация о преобразованиях приведена в разделе 4.11 (стр. 218).

Перегруженные операторы

Значение операторов для встроенных и составных типов определяет сам язык. Значение большинства операторов типов классов мы можем определить самостоятельно. Поскольку такие определения придают альтернативное значение существующему символу оператора, они называются *перегруженными операторами* (overloaded operator). Операторы `>>` и `<<` библиотеки ввода и вывода, а также операторы, использовавшиеся с объектами строк, векторов и итераторов, являются перегруженными операторами.

При использовании перегруженного оператора его смысл, а также тип операндов и результата зависят от того, как определен оператор. Однако количество операндов, их приоритет и порядок не могут быть изменены.



L- и r-значения

Каждое выражение в языке C++ является либо *r-значением* (r-value), либо *l-значением* (l-value). Эти названия унаследованы от языка C и первоначально имели простую mnemonicическую цель: l-значения могли стоять слева от оператора присвоения, а r-значения не могли.

В языке C++ различие не так просто. В языке C++ выражение l-значения возвращает объект или функцию. Однако некоторые l-значения, такие как константные объекты, не могут быть левым операндом присвоения. Кроме того, некоторые выражения возвращают объекты, но возвращают их как r-, а не l-значения. Короче говоря, при применении объекта в качестве r-значения используется его значение (т.е. его содержимое). При применении объекта в качестве l-значения используется его идентификатор (т.е. его область в памяти).

Операторы различаются по тому, требуют ли они операндов l- или r-значения, а также по тому, возвращают ли они l- или r-значения. Важный момент здесь в том, что (за одним исключением, рассматриваемым в разделе 13.6 (стр. 672)) l-значение можно использовать там, где требуется r-значение, однако нельзя использовать r-значение там, где требуется l-значение (т.е. область). Когда l-значение применяется вместо r-значения, используется содержимое объекта (его значение). Мы уже использовали несколько операторов, которые действовали l-значения.

- В качестве своего левого операнда оператор присвоения требует (неконстантного) l-значения и возвращает свой левый операнд как l-значение.
- Оператор обращения к адресу (см. раздел 2.3.2, стр. 87) требует в качестве операнда l-значение и возвращает указатель на свой операнд как r-значение.

- Встроенные операторы обращения к значению и индексирования (см. раздел 2.3.2, стр. 89 и раздел 3.5.2, стр. 165), а также обращение к значению итератора и операторы индексирования строк и векторов (см. раздел 3.4.1, стр. 153, раздел 3.2.3, стр. 138 и раздел 3.3.3, стр. 148) возвращают l-значения.
- Операторы инкремента и декремента, как встроенные, так и итератора (см. раздел 1.4.1, стр. 38 и раздел 3.4.1, стр. 155), требуют l-значения в качестве операндов. Их префиксные версии (которые использовались до сих пор) также возвращают l-значения.

Рассматривая операторы, следует обратить внимание на то, должен ли операнд быть l-значением и возвращает ли он l-значение.

L- и r-значения также различаются при использовании спецификатора `decltype` (см. раздел 2.5.3, стр. 110). При применении спецификатора `decltype` к выражению (отличному от переменной) результатом будет ссылочный тип, если выражение возвращает l-значение. Предположим, например, что указатель `p` имеет тип `int*`. Поскольку обращение к значению возвращает l-значение, выражение `decltype(*p)` имеет тип `int&`. С другой стороны, поскольку оператор обращения к адресу возвращает r-значение, выражение `decltype(&p)` имеет тип `int**`, т.е. указатель на указатель на тип `int`.



4.1.2. Приоритет и порядок

Выражения с двумя или несколькими операторами называются *составными* (compound expression). Результат составного выражения определяет способ группировки операндов в отдельных операторах. Группировку операндов определяют приоритет и порядок. Таким образом, они определяют, какие части выражения будут операндами для каждого из операторов в выражении. При помощи скобок программисты могут изменять эти правила, обеспечивая необходимую группировку.

Обычно значение выражения зависит от того, как группируются его части. Операнды операторов с более высоким приоритетом группируются прежде операндов операторов с более низким приоритетом. Порядок определяет то, как группируются операнды с тем же приоритетом. Например, операторы умножения и деления имеют одинаковый приоритет относительно друг друга, но их приоритет выше, чем у операторов сложения и вычитания. Поэтому операнды операторов умножения и деления группируются прежде операндов операторов сложения и вычитания. Арифметические операторы имеют левосторонний порядок, т.е. они группируются слева направо.

- Благодаря приоритету результатом выражения $3+4*5$ будет 23, а не 35.
- Благодаря порядку результатом выражения $20-15-3$ будет 2, а не 8.

В более сложном примере при вычислении слева направо следующего выражения получается 20:

$$6 + 3 * 4 / 2 + 2$$

Вполне возможны и другие результаты: 9, 14 и 36. В языке C++ результат составит 14, поскольку это выражение эквивалентно следующему:

```
// здесь круглые скобки соответствуют стандартному приоритету и порядку
((6 + ((3 * 4) / 2)) + 2)
```

Круглые скобки переопределяют приоритет и порядок

Круглые скобки позволяют переопределить обычную группировку. Выражения в круглых скобках обрабатываются как отдельные модули, а во всех остальных случаях применяются обычные правила приоритета. Например, используя круглые скобки в предыдущем выражении, можно принудительно получить любой из четырех возможных вариантов:

```
// круглые скобки обеспечивают альтернативные группировки
cout << (6 + 3) * (4 / 2 + 2) << endl; // выводит 36
cout << ((6 + 3) * 4) / 2 + 2 << endl; // выводит 20
cout << 6 + 3 * 4 / (2 + 2) << endl; // выводит 9
```



Когда важны приоритет и порядок

Мы уже видели примеры, где приоритет влияет на правильность наших программ. Рассмотрим обсуждавшийся в разделе 3.5.3 (стр. 171) пример обращения к значению и арифметических действий с указателями.

```
int ia[] = {0,2,4,6,8}; // массив из 5 элементов типа int
int last = *(ia + 4);   // ok: инициализирует last значением ia[4], т.е. 8
last = *ia + 4;         // last = 4, эквивалент ia[0] + 4
```

Если необходим доступ к элементу в области `ia+4`, то круглые скобки вокруг сложения необходимы. Без круглых скобок сначала группируется часть `*ia`, а к полученному значению добавляется 4.

Наиболее популярный случай, когда порядок имеет значение, — это выражения ввода и вывода. Как будет продемонстрировано в разделе 4.8 (стр. 214), операторы ввода и вывода имеют левосторонний порядок. Этот порядок означает, что можно объединить несколько операций ввода и вывода в одном выражении.

```
cin >> v1 >> v2; // читать в v1, а затем в v2
```

В таблице раздела 4.12 (стр. 227) перечислены все операторы, организованные по сегментам, отделенным линиями. У операторов в каждом сегменте одинаковый приоритет, причем сегменты с более высоким приоритетом расположены выше. Например, префиксный оператор инкремента и оператор обращения к значению имеют одинаковый приоритет, который выше, чем та-

ковой у арифметических операторов. Таблица содержит ссылки на страницы, где описан каждый из операторов. Многие из этих операторов уже применялось, а большинство из остальных рассматривается в данной главе. Подробней некоторые из операторов рассматриваются позже.

Упражнения раздела 4.1.2

Упражнение 4.1. Какое значение возвратит выражение $5 + 10 * 20/2$?

Упражнение 4.2. Используя таблицу раздела 4.12 (стр. 214), расставьте скобки в следующих выражениях, чтобы обозначить порядок группировки операндов:

(a) `* vec.begin()` (b) `* vec.begin() + 1`



4.1.3. Порядок вычисления

Приоритет определяет группировку операндов. Но он ничего не говорит о порядке, в котором обрабатываются операнды. В большинстве случаев порядок не определен. В следующем выражении известно, что функции `f1()` и `f2()` будут вызваны перед умножением:

```
int i = f1() * f2();
```

В конце концов, умножаются именно их результаты. Тем не менее нет никакого способа узнать, будет ли функция `f1()` вызвана до функции `f2()`, или наоборот.

Для операторов, которые не определяют порядок вычисления, выражение, пытающееся *обратиться к тому же объекту и изменить его*, было бы ошибочным. Выражения, которые действительно так поступают, имеют непредсказуемое поведение (см. раздел 2.1.2, стр. 67). Вот простой пример: оператор `<<` не дает никаких гарантий в том, как и когда обрабатываются его операнды. В результате следующее выражение вывода непредсказуемо:

```
int i = 0;
cout << i << " " << ++i << endl; // непредсказуемо
```

Непредсказуемость этой программы в том, что нет никакой возможности сделать выводы о ее поведении. Компилятор мог бы сначала обработать часть `++i`, а затем часть `i`, тогда вывод будет `1 1`. Но компилятор мог бы сначала обработать часть `i`, тогда вывод будет `0 1`. Либо компилятор мог бы сделать что-то совсем другое. Поскольку у этого выражения неопределенное поведение, программа ошибочна, независимо от того, какой код создает компилятор.

Четыре оператора действительно гарантируют порядок обработки операндов. В разделе 3.2.3 (стр. 138) упоминалось о том, что оператор логического AND (`&&`) гарантирует выполнение сначала левого операнда. Кроме того, он

гарантирует, что правый операнд обрабатывается только при истинности левого операнда. Другими операторами, гарантирующими порядок обработки операндов, являются оператор логического OR (`||`) (раздел 4.3, стр. 197), условный оператор (`? :`) (раздел 4.7, стр. 208) и оператор запятая (`,`) (раздел 4.10, стр. 217).



Порядок вычисления, приоритет и порядок операторов

Порядок вычисления операндов не зависит от приоритета и порядка операторов. Рассмотрим следующее выражение:

`f() + g() * h() + j()`

- Приоритет гарантирует умножение результатов вызова функций `g()` и `h()`.
- Порядок гарантирует добавление результата вызова функции `f()` к произведению `g()` и `h()`, а также добавление результата сложения к результату вызова функции `j()`.
- Однако нет никаких гарантий относительно порядка вызова этих функций.

Если функции `f()`, `g()`, `h()` и `j()` являются независимыми и не влияют на состояние тех же объектов или выполняют ввод и вывод, то порядок их вызова несуществен. Но если любые из этих функций действительно воздействуют на тот же объект, то выражение ошибочно, а его поведение непредсказуемо.

Упражнения раздела 4.1.3

Упражнение 4.3. Порядок вычисления большинства парных операторов остается неопределенным, чтобы предоставить компилятору возможность для оптимизации. Эта стратегия является компромиссом между созданием эффективного кода и потенциальными проблемами в использовании языка программистом. Полагаете этот компромисс приемлемым? Кто-то да, кто-то нет.

Совет. Манипулирование составными выражениями

При написании составных выражений могут пригодиться два эмпирических правила.

1. В сомнительных случаях заключайте выражения в круглые скобки, чтобы явно сгруппировать операнды в соответствии с логикой программы.
2. При изменении значения операнда не используйте этот операнд в другом месте того же оператора.

Важнейшим исключением из второго правила является случай, когда часть выражения, изменяющая операнд, сама является операндом другой части выражения. Например, в выражении `*++iter` инкремент изменяет значение итератора `iter`, а измененное значение используется как операнд оператора `*`. В этом и подобных выражениях порядок обработки операндов не является проблемным. Но в больших выражениях те части, которые изменяют операнд, должны обрабатываться в первую очередь. Такой подход не создает никаких проблем и применяется достаточно часто.

4.2. Арифметические операторы

Таблица 4.1. Арифметические операторы (левосторонний порядок)

Оператор	Действие	Применение
+	Унарный плюс	+ <i>выражение</i>
-	Унарный минус	- <i>выражение</i>
*	Умножение	<i>выражение</i> * <i>выражение</i>
/	Деление	<i>выражение</i> / <i>выражение</i>
%	Остаток	<i>выражение</i> % <i>выражение</i>
+	Сложение	<i>выражение</i> + <i>выражение</i>
-	Вычитание	<i>выражение</i> - <i>выражение</i>

В табл. 4.1 (и таблицах операторов последующих разделов) операторы сгруппированы по приоритету. Унарные арифметические операторы имеют более высокий приоритет, чем операторы умножения и деления, которые в свою очередь имеют более высокий приоритет, чем парные операторы вычитания и сложения. Операторы с более высоким приоритетом группируются перед операторами с более низким приоритетом. Все эти операторы имеют левосторонний порядок, т.е. при равенстве приоритетов они группируются слева направо.

Если не указано иное, то арифметические операторы могут быть применены к любому арифметическому типу (см. раздел 2.1.1, стр. 62) или любому типу, который может быть преобразован в арифметический тип. Операнды и результаты этих операторов являются *г*-значениями. Как упоминается в разделе 4.11 (стр. 218), в ходе вычисления операторов их операнды малых целочисленных типов преобразуются в больший целочисленный тип и все операнды могут быть преобразованы в общий тип.

Унарные операторы плюс и минус могут быть также применены к указателям. Использование парных операторов `+` и `-` с указателями рассматривалось

в разделе 3.5.3 (стр. 169). Будучи примененным к указателю или арифметическому значению, унарный плюс возвращает (возможно, преобразованную) копию значения своего операнда.

Унарный оператор минус возвращает отрицательную копию (возможно, преобразованную) значения своего операнда.

```
int i = 1024;
int k = -i;    // i равно -1024
bool b = true;
bool b2 = -b; // b2 равно true!
```

В разделе 2.1.1 (стр. 64) упоминалось, что значения типа `bool` не нужно использовать для вычислений. Результат `-b` — хороший пример того, что имелось в виду.

Для большинства операторов операнды типа `bool` преобразуются в тип `int`. В данном случае значение переменной `b`, `true`, преобразуется в значение `1` типа `int` (см. раздел 2.1.2, стр. 66). Это (преобразованное) значение преобразуется в отрицательное, `-1`. Значение `-1` преобразуется обратно в тип `bool` и используется для инициализации переменной `b2`. Поскольку значение инициализатора отлично от нуля, при преобразовании в тип `bool` его значением станет `true`. Таким образом, значением `b2` будет `true`!

ВНИМАНИЕ! ПЕРЕПОЛНЕНИЕ ПЕРЕМЕННОЙ И ДРУГИЕ АРИФМЕТИЧЕСКИЕ ОСОБЕННОСТИ

Некоторые арифметические выражения возвращают неопределенный результат. Некоторые из этих неопределенностей имеют математический характер, например деление на нуль. Причиной других являются особенности компьютеров, например, переполнение, происходящее при превышении вычисленным значением размера области памяти, представленной его типом.

Предположим, тип `short` занимает на машине 16 битов. В этом случае переменная типа `short` способна хранить максимум значение 32767. На такой машине следующий составной оператор присвоения приводит к переполнению.

```
short short_value = 32767; // максимальное значение при short 16 битов
short_value += 1;          // переполнение
cout << "short_value: " << short_value << endl;
```

Результат присвоения `1` переменной `short_value` непредсказуем. Для хранения знакового значения 32768 требуется 17 битов, но доступно только 16. Многие системы никак не предупреждают о переполнении ни во время выполнения, ни во время компиляции. Подобно любой ситуации с неопределенностью, результат оказывается непредсказуем. На системе авторов программа завершилась с таким сообщением:

```
short_value: -32768
```

Здесь произошло переполнение переменной: предназначенный для знака разряд содержал значение 0, но был заменен на 1, что привело к появлению отрицательного значения. На другой системе результат мог бы быть иным, либо программа могла бы повести себя по-другому, включая полный отказ.

Примененные к объектам арифметических типов, операторы $+$, $-$, $*$ и $/$ имеют вполне очевидные значения: сложение, вычитание, умножение и деление. Результатом деления целых чисел является целое число. Получаемая в результате деления дробная часть отбрасывается.

```
int ival1 = 21/6; // ival1 равно 3; результат усекается;
                // остаток отбрасывается
int ival2 = 21/7; // ival2 равно 3; остатка нет;
                // результат - целочисленное значение
```

Оператор $\%$ известен как *остаток* (remainder), или *оператор деления по модулю* (modulus). Он позволяет вычислить остаток от деления левого операнда на правый. Его операнды должны иметь целочисленный тип.

```
int ival = 42;
double dval = 3.14;
ival % 12; // ok: возвращает 6
ival % dval; // ошибка: операнд с плавающей запятой
```

C++ 11 При делении отличное от нуля частное позитивно, если у операндов одинаковый знак, и отрицательное в противном случае. Прежние версии языка разрешали округление отрицательного частного вверх или вниз; однако новый стандарт требует округления частного до нуля (т.е. усечения).

Оператор деления по модулю определен так, что если m и n целые числа и n отлично от нуля, то $(m/n) * n + m \% n$ равно m . По определению, если $m \% n$ отлично от нуля, то у него тот же знак, что и у m . Прежние версии языка разрешали результату выражения $m \% n$ иметь тот же знак, что и у n , причем на реализациях, у которых отрицательный результат выражения m/n округлялся не до нуля, но такие реализации сейчас запрещены. Кроме того, за исключением сложного случая, где $-m$ приводит к переполнению, $(-m)/n$ и $m/(-n)$ всегда эквивалентны $-(m/n)$, $m \% (-n)$ эквивалентно $m \% n$ и $(-m) \% n$ эквивалентно $-(m \% n)$. А конкретно:

```
21 % 6; /* результат 3 */      21 / 6; /* результат 3 */
21 % 7; /* результат 0 */      21 / 7; /* результат 3 */
-21 % -8; /* результат -5 */   -21 / -8; /* результат 2 */
21 % -5; /* результат 1 */      21 / -5; /* результат -4 */
```

Упражнения раздела 4.2

Упражнение 4.4. Расставьте скобки в следующем выражении так, чтобы продемонстрировать порядок его обработки. Проверьте свой ответ, откомпилировав и отобразив результат выражения без круглых скобок.

$12 / 3 * 4 + 5 * 15 + 24 \% 4 / 2$

Упражнение 4.5. Определите результат следующих выражений:

(a) $-30 * 3 + 21 / 5$ (b) $-30 + 3 * 21 / 5$

(c) $30 / 3 * 21 \% 5$ (d) $-30 / 3 * 21 \% 4$

Упражнение 4.6. Напишите выражение, чтобы определить, является ли значение типа `int` четным или нечетным.

Упражнение 4.7. Что значит переполнение? Представьте три выражения, приводящих к переполнению.

4.3. Логические операторы и операторы отношения

Операторам отношения передают операторы арифметического типа или типа указателя, а логическим операторам — операнды любого типа, допускающего преобразование в тип `bool`. Все они возвращают значение типа `bool`. Арифметические операнды и указатели со значением нуль рассматриваются как значение `false`, а все другие как значение `true`. Операнды для этих операторов являются r-значениями, а результат — r-значение.

Таблица 4.2. Логические операторы и операторы отношения

Порядок	Оператор	Действие	Применение
Правосторонний	<code>!</code>	Логическое NOT	<code>! выражение</code>
Левосторонний	<code><</code>	Меньше	<code>выражение < выражение</code>
Левосторонний	<code><=</code>	Меньше или равно	<code>выражение <= выражение</code>
Левосторонний	<code>></code>	Больше	<code>выражение > выражение</code>
Левосторонний	<code>>=</code>	Больше или равно	<code>выражение >= выражение</code>
Левосторонний	<code>==</code>	Равно	<code>выражение == выражение</code>
Левосторонний	<code>!=</code>	Не равно	<code>выражение != выражение</code>
Левосторонний	<code>&&</code>	Логическое AND	<code>выражение && выражение</code>
Левосторонний	<code> </code>	Логическое OR	<code>выражение выражение</code>

Операторы логического AND и OR

Общим результатом оператора логического AND (&&) является `true`, если и только если оба его операнда рассматриваются как `true`. Оператор логического OR (||) возвращает значение `true`, если любой из его операндов рассматривается как `true`.

Операторы логического AND и OR всегда обрабатывают свой левый операнд перед правым. Кроме того, правый операнд обрабатывается, *если и только если* левый операнд не определил результат. Эта стратегия известна как *вычисление по сокращенной схеме* (short-circuit evaluation).

- Правая сторона оператора && вычисляется, если и только если левая сторона истинна.
- Правая сторона оператора || вычисляется, если и только если левая сторона ложна.

Оператор логического AND использовался в некоторых из программ главы 3. Эти программы использовали левый операнд для проверки, безопасно ли выполнять правый операнд. Например, условие цикла `for` на стр. 139: сначала проверялось, что `index` не достиг конца строки:

```
index != s.size() && !isspace(s[index])
```

Это гарантировало, что правый операнд не будет выполнен, если индекс уже вышел из диапазона.

Рассмотрим пример применения оператора логического OR. Предположим, что в векторе `str` имеется некоторый текст, который необходимо вывести, добавляя символ новой строки после каждой пустой строки или после строки, завершающейся точкой. Для отображения каждого элемента используем серийный оператор `for` (раздел 3.2.3, стр. 138):

```
// обратите внимание, s - ссылка на константу; элементы не копируются и
// не могут быть изменены
for (const auto &s : text) { // для каждого элемента text
    cout << s;               // вывести текущий элемент
    // пустые строки и строки, завершающиеся точкой, требуют новой строки
    if (s.empty() || s[s.size() - 1] == '.')
        cout << endl;
    else
        cout << " ";        // в противном случае отделить пробелом
}
```

После вывода текущего элемента выясняется, есть ли необходимость вывести новую строку. Условие оператора `if` сначала проверяет, не пуста ли строка `s`. Если это так, то необходимо вывести новую строку независимо от значения правого операнда. Только если строка не пуста, обрабатывается второе выражение, которое проверяет, не заканчивается ли строка точкой. Это

выражение полагается на вычисление по сокращенной схеме оператора `||`, гарантирующего индексирование строки `s`, только если она не пуста.

Следует заметить, что переменная `s` объявлена как ссылка на константу (см. раздел 2.5.2, стр. 108). Элементами вектора `text` являются строки, и они могут быть очень большими, а использование ссылки позволяет избежать их копирования. Поскольку запись в элементы не нужна, объявляем `s` ссылкой на константу.

Оператор логического NOT

Оператор логического NOT (!) возвращает инверсию исходного значения своего операнда. Этот оператор уже использовался в разделе 3.2.2 (стр. 130). В следующем примере подразумевается, что `vec` — это вектор целых чисел, для проверки наличия значений в элементах которого используется оператор логического NOT для значения, возвращенного функцией `empty()`.

```
// отобразить первый элемент вектора vec, если он есть
if (!vec.empty())
    cout << vec[0];
```

Подвыражение `!vec.empty()` возвращает значение `true`, если вызов функции `empty()` возвращает значение `false`.

Операторы отношения

Операторы отношения (`<`, `<=`, `>`, `>=`) имеют свой обычный смысл и возвращают значение типа `bool`. Эти операторы имеют левосторонний порядок.

Поскольку операторы отношения возвращают логическое значение, их сцепление может дать удивительный результат:

```
// Упс! это условие сравнивает k с результатом сравнения i < j
if (i < j < k) // true, если k больше 1!
```

Условие группирует `i` и `j` в первый оператор `<`. Результат этого выражения (типа `bool`) является левым операндом второго оператора `<`. Таким образом, переменная `k` сравнивается с результатом (`true` или `false`) первого оператора сравнения! Для реализации той проверки, которая и предполагалась, выражение нужно переписать следующим образом:

```
// условие истинно, если i меньше, чем j, и j меньше, чем k
if (i < j && j < k) { /* ... */ }
```

Проверка равенства и логические литералы

Если необходимо проверить истинность арифметического значения или объекта указателя, то самый простой способ подразумевает использование этого значения как условия.

```
if (val) { /* ... */ } // true, если val - любое не нулевое значение
if (!val) { /* ... */ } // true, если val - ноль
```

В обоих условиях компилятор преобразовывает `val` в тип `bool`. Первое условие истинно, пока значение переменной `val` отлично от нуля; второе истинно, если `val` — ноль.

Казалось бы, условие можно переписать так:

```
if (val == true) { /* ... */ } // true, только если val равно 1!
```

У этого подхода две проблемы. Прежде всего, он длинней и менее непосредствен, чем предыдущий код (хотя по общему признанию в начале изучения языка C++ этот код понятней). Но важней всего то, что если тип переменной `val` отличен от `bool`, то это сравнение работает не так, как ожидалось.

Если переменная `val` имеет тип, отличный от `bool`, то перед применением оператора `==` значение `true` преобразуется в тип переменной `val`. Таким образом, получается код, аналогичный следующему:

```
if (val == 1) { /* ... */ }
```

Как уже упоминалось, при преобразовании значения типа `bool` в другой арифметический тип `false` преобразуется в 0, а `true` — в 1 (см. раздел 2.1.2, стр. 66). Если бы нужно было действительно сравнить значение переменной `val` со значением 1, то условие так и следовало бы написать.



ВНИМАНИЕ

Использование логических литералов `true` и `false` в качестве операндов сравнения — обычно плохая идея. Эти литералы следует использовать только для сравнения с объектами типа `bool`.

Упражнения раздела 4.3

Упражнение 4.8. Объясните, когда обрабатываются операнды операторов логического AND, логического OR и оператора равенства.

Упражнение 4.9. Объясните поведение следующего условия оператора `if`:

```
const char *cp = "Hello World";
if (cp && *cp)
```

Упражнение 4.10. Напишите условие цикла `while`, который читал бы целые числа со стандартного устройства ввода, пока во вводе не встретится значение 42.

Упражнение 4.11. Напишите выражение, проверяющее четыре значения `a`, `b`, `c` и `d` и являющееся истинным, если значение `a` больше `b`, которое больше `c`, которое больше `d`.

Упражнение 4.12. С учетом того, что `i`, `j` и `k` имеют тип `int`, объясните значение выражения `i != j < k`.

4.4. Операторы присвоения

Левым операндом оператора присвоения должно быть допускающее изменение l-значение. Ниже приведено несколько примеров недопустимых попыток присвоения.

```
int i = 0, j = 0, k = 0; // инициализация, а не присвоение
const int ci = i;        // инициализация, а не присвоение
1024 = k; // ошибка: литерал является r-значением
i + j = k; // ошибка: арифметическое выражение - тоже r-значение
ci = k;    // ошибка: ci - константа (неизменяемое l-значение)
```

Результат присвоения, левый операнд, является l-значением. Тип результата совпадает с типом левого операнда. Если типы левого и правого операндов отличаются, тип правого операнда преобразуется в тип левого.

```
k = 0; // результат: тип int, значение 0
k = 3.14159; // результат: тип int, значение 3
```

**C++
11** По новому стандарту с правой стороны можно использовать список инициализации (см. раздел 2.2.1, стр. 77):

```
k = {3.14}; // ошибка: сужающее преобразование
vector<int> vi; // первоначально пусто
vi = {0,1,2,3,4,5,6,7,8,9}; // теперь vi содержит десять элементов
// со значениями от 0 до 9
```

Если левый операнд имеет встроенный тип, список инициализации может содержать максимум одно значение, и это значение не должно требовать сужающего преобразования (narrowing conversion) (см. раздел 2.2.1, стр. 77).

Для типов классов происходящее зависит от подробностей класса. В случае вектора шаблон `vector` определяет собственную версию оператора присвоения, позволяющего использовать список инициализации. Этот оператор заменяет элементы вектора с левой стороны элементами списка с правой.

Независимо от типа левого операнда список инициализации может быть пуст. В данном случае компилятор создает инициализированный значением по умолчанию (см. раздел 3.3.1, стр. 143) временный объект и присваивает это значение левому операнду.

Оператор присвоения имеет правосторонний порядок

В отличие от других парных операторов, присвоение имеет правосторонний порядок:

```
int ival, jval;
ival = jval = 0; // ok: каждой переменной присвоено значение 0
```

Поскольку присвоение имеет правосторонний порядок, его крайняя правая часть, `jval = 0`, является правым операндом крайнего левого оператора

присвоения. Поскольку присвоение возвращает свой левый операнд, результат крайнего правого присвоения (т.е. `jval`) присваивается переменной `ival`.

Каждый объект в множественном операторе присвоения должен иметь тип, совпадающий с типом соседа справа, или допускать преобразование в него (раздел 4.11, стр. 218):

```
int ival, *pval; // ival имеет тип int; pval имеет тип указателя на int
ival = pval = 0; // ошибка: переменной типа int нельзя присвоить
                  // значение указателя
string s1, s2;
s1 = s2 = "OK"; // строковый литерал "OK" преобразован в строку
```

Первое присвоение некорректно, поскольку объекты `ival` и `pval` имеют разные типы и не существует преобразования типа `int*` (`pval`) в тип `int` (`ival`). Оно некорректно, несмотря на то, что значение нуль может быть присвоено любому объекту.

Второе присвоение, напротив, вполне допустимо. Строковый литерал преобразуется в значение типа `string`, которое и присваивается переменной `s2` типа `string`. Результат этого присвоения — строка `s2` — имеет тот же тип, что и строка `s1`.

Оператор присвоения имеет низкий приоритет

Присвоения нередко происходят в условиях. Поскольку оператор присвоения имеет относительно низкий приоритет, его обычно заключают в скобки, чтобы он работал правильно. Чтобы продемонстрировать, чем присвоение может быть полезно в условии, рассмотрим следующий цикл. Здесь необходимо вызывать функцию до тех пор, пока она не возвратит желаемое значение, скажем 42.

```
// подробный, а потому более подверженный ошибкам способ написания цикла
int i = get_value(); // получить первое значение
while (i != 42) {
    // выполнить действия ...
    i = get_value(); // получить остальные значения
}
```

Код начинается с вызова функции `get_value()`, затем следует цикл, условие которого использует значение, возвращенное этим вызовом. Последним оператором этого цикла является еще один вызов функции `get_value()`, далее цикл повторяется. Этот код можно переписать более непосредственно:

```
int i;
// лучший вариант цикла, теперь вполне понятно, что делает условие
while ((i = get_value()) != 42) {
    // выполнить действия ...
}
```

Теперь условие вполне однозначно выражает намерение разработчика: необходимо продолжать, пока функция `get_value()` не возвратит значение 42. В ходе вычисления условия результат вызова функции `get_value()` присваивается переменной `i`, значение которой затем сравнивается со значением 42.

Без круглых скобок операндами оператора `!=` было бы значение, возвращенное функцией `get_value()` и 42, а результат проверки (`true` или `false`) был бы присвоен переменной `i`, чего явно не планировалось!



Поскольку приоритет оператора присвоения ниже, чем у операторов отношения, круглые скобки вокруг присвоений в условиях обычно необходимы.

Не перепутайте операторы равенства и присвоения

Тот факт, что присвоение возможно в условии, зачастую имеет удивительные последствия:

```
if (i = j)
```

Условие оператора `if` присваивает значение переменной `j` переменной `i`, а затем проверяет результат присвоения. Если значение переменной `j` отлично от нуля, то условие истинно. Однако автор этого кода почти наверняка намеревался проверить равенство значений переменных `i` и `j` так:

```
if (i == j)
```

Ошибки такого рода хоть и известны, но трудны для обнаружения. Некоторые, но не все компиляторы достаточно “любезны”, чтобы предупредить о таком коде, как в этом примере.

Составные операторы присвоения

Довольно нередки случаи, когда оператор применяется к объекту, а полученный результат повторно присваивается тому же объекту. В качестве примера рассмотрим программу из раздела 1.4.2 (стр. 40):

```
int sum = 0;
// сложить числа от 1 до 10 включительно
for (int val = 1; val <= 10; ++val)
    sum += val; // эквивалентно sum = sum + val
```

Подобный вид операций характерен не только для сложения, но и для других арифметических и побитовых операторов, которые рассматриваются в разделе 4.8 (стр. 210). Соответствующие *составные операторы присвоения* (compound assignment) существуют для каждого из этих операторов.

```
+=      -=      *=      /=      %=    // арифметические операторы
<<=    >>=      &=      ^=      |=    // побитовые операторы; см. р 4.8 (стр. 210)
```

Каждый составной оператор по сути эквивалентен обычному, за исключением того, что, когда используется составное присвоение, левый операнд обрабатывается (оценивается) только однажды.

Но эти формы имеют одно очень важное различие: в составном операторе присвоения левый операнд вычисляется только один раз. По существу, он эквивалентен следующему:

```
a = a оператор b;
```

Если используется обычное присвоение, операнд *a* обрабатывается дважды: один раз в выражении с правой стороны и во второй раз — как операнд слева. В подавляющем большинстве случаев это различие несущественно, возможно, кроме тех, где критически важна производительность.

Упражнения раздела 4.4

Упражнение 4.13. Каковы значения переменных *i* и *d* после каждого присвоения?

```
int i; double d;
(a) d = i = 3.5; (b) i = d = 3.5;
```

Упражнение 4.14. Объясните, что происходит в каждом из следующих операторов *if*?

```
if (42 = i) // ...
if (i = 42) // ...
```

Упражнение 4.15. Следующее присвоение недопустимо. Почему? Как исправить ситуацию?

```
double dval; int ival; int *pi;
dval = ival = pi = 0;
```

Упражнение 4.16. Хотя ниже приведены вполне допустимые выражения, их поведение может оказаться не таким, как предполагалось. Почему? Перепишите выражения так, чтобы они стали более понятными.

```
(a) if (p = getPtr() != 0) (b) if (i = 1024)
```

4.5. Операторы инкремента и декремента

Операторы инкремента (*++*) и декремента (*--*) позволяют в краткой и удобной форме добавить или вычесть единицу из объекта. Эта форма записи обеспечивает не только удобство, она весьма популярна при работе с итераторами, поскольку большинство итераторов не поддерживает арифметических действий.

Эти операторы существуют в двух формах: префиксной и постфиксной. До сих пор использовался только *префиксный оператор инкремента* (*prefix increment*). Он осуществляет инкремент (или декремент) своего операнда и возвращает *измененный* объект как результат. *Постфиксный оператор инкремента*

ma (postfix increment) (или декремента) возвращает копию первоначального операнда *неизменной*, а затем изменяет значение операнда.

```
int i = 0, j;
j = ++i; // j = 1, i = 1: префикс возвращает увеличенное значение
j = i++; // j = 1, i = 2: постфикс возвращает исходное значение
```

Операндами этих операторов должны быть l-значения. Префиксные операторы возвращают сам объект как l-значение. Постфиксные операторы возвращают копию исходного значения объекта как r-значение.

СОВЕТ. ИСПОЛЬЗУЙТЕ ПОСТФИКСНЫЕ ОПЕРАТОРЫ ТОЛЬКО ПО МЕРЕ НЕОБХОДИМОСТИ

Читатели с опытом языка C могли бы быть удивлены тем, что в написанных до сих пор программах использовался префиксный оператор инкремента. Причина проста: префиксная версия позволяет избежать ненужной работы. Она увеличивает значение и возвращает результат. Постфиксный оператор должен хранить исходное значение, чтобы вернуть неувеличенное значение как результат. Но если в исходном значении нет никакой потребности, то нет необходимости и в дополнительных действиях, осуществляемых постфиксным оператором.

Для переменных типа `int` и указателей компилятор способен оптимизировать код и уменьшить количество дополнительных действий. Для более сложных типов итераторов подобные дополнительные действия могут обойтись довольно дорого. При использовании префиксных версий об эффективности можно не волноваться. Кроме того, а возможно и важнее всего то, что так можно выразить свои намерения более непосредственно.



Объединение операторов обращения к значению и инкремента в одном выражении

Постфиксные версии операторов `++` и `--` используются в случае, когда в одном составном выражении необходимо использовать текущее значение переменной, а затем увеличить его.

В качестве примера используем постфиксный оператор инкремента для написания цикла, выводящего значения вектора до, но не включая, первого отрицательного значения.

```
auto pbeg = v.begin();
// отображать элементы до первого отрицательного значения
while (pbeg != v.end() && *pbeg >= 0)
    cout << *pbeg++ << endl; // отобразить текущее значение и
                             // переместить указатель pbeg
```

Выражение `*pbeg++` обычно малопонятно новичкам в языках C++ и C. Но поскольку эта схема весьма распространена, программисты C++ должны понимать такие выражения.

Приоритет постфиксного оператора инкремента выше, чем оператора обращения к значению, поэтому код `*pbeg++` эквивалентен коду `*(pbeg++)`. Часть `pbeg++` осуществляет инкремент указателя `pbeg` и возвращает как результат копию предыдущего значения указателя `pbeg`. Таким образом, операндом оператора `*` будет неувеличенное значение указателя `pbeg`. Следовательно, оператор выводит элемент, на который первоначально указывал указатель `pbeg`, а затем осуществляет его инкремент.

Этот подход основан на том, что постфиксный оператор инкремента возвращает копию своего исходного, не увеличенного операнда. Если бы он возвратил увеличенное значение, то обращение к элементу вектора по такому увеличенному значению привело бы к плачевным результатам: первым оказался бы незаписанный элемент вектора. Хуже того, если бы у последовательности не было никаких отрицательных значений, то в конце произошла бы попытка обращения к значению несуществующего элемента за концом вектора.

СОВЕТ. КРАТКОСТЬ МОЖЕТ БЫТЬ ДОСТОИНСТВОМ

Такие выражения, как `*iter++`, могут быть не очевидны, однако они весьма популярны. Следующая форма записи проще и менее подвержена ошибкам:

```
cout << *iter++ << endl;
```

чем ее более подробный эквивалент:

```
cout << *iter << endl;  
++iter;
```

Поэтому примеры подобного кода имеет смысл внимательно изучать, чтобы они стали совершенно понятны. В большинстве программ C++ используются краткие формы выражений, а не их более подробные эквиваленты. Поэтому программистам C++ придется привыкать к ним. Кроме того, научившись работать с краткими формами, можно заметить, что они существенно менее подвержены ошибкам.

Помните, что операнды могут быть обработаны в любом порядке

Большинство операторов не гарантирует последовательности обработки операндов (см. раздел 4.1.3, стр. 192). Отсутствие гарантированного порядка зачастую не имеет значения. Это действительно имеет значение в случае, когда выражение одного операнда изменяет значение, используемое выражением другого. Поскольку операторы инкремента и декремента изменяют свои операнды, очень просто неправильно использовать эти операторы в составных выражениях.

Для иллюстрации проблемы перепишем цикл из раздела 3.4.1 (стр. 155), который преобразует в верхний регистр символы первого введенного слова:

```
for (auto it = s.begin(); it != s.end() && !isspace(*it); ++it)
    *it = toupper(*it); // преобразовать в верхний регистр
```

Этот пример использует цикл `for`, позволяющий отделить оператор обращения к значению `beg` от оператора его приращения. Замена цикла `for`, казалось бы, эквивалентным циклом `while` дает неопределенные результаты:

```
// поведение следующего цикла неопределенно!
while (beg != s.end() && !isspace(*beg))
    *beg = toupper(*beg++); // ошибка: это присвоение неопределенно
```

Проблема пересмотренной версии в том, что левый и правый операнды оператора `=` используют значение, на которое указывает `beg`, и правый его изменяет. Поэтому присвоение неопределенно. Компилятор мог бы обработать это выражение так:

```
*beg = toupper(*beg); // сначала обрабатывается левая сторона
*(beg + 1) = toupper(*beg); // сначала обрабатывается правая сторона
```

Или любым другим способом.

Упражнения раздела 4.5

Упражнение 4.17. Объясните различие между префиксным и постфиксным инкрементом.

Упражнение 4.18. Что будет, если цикл `while` на странице стр. 207, используемый для отображения элементов вектора, задействует префиксный оператор инкремента?

Упражнение 4.19. С учетом того, что `ptr` указывает на тип `int`, `vec` — вектор `vector<int>`, а `ival` имеет тип `int`, объясните поведение каждого из следующих выражений. Есть ли среди них неправильные? Почему? Как их исправить?

- (a) `ptr != 0 && *ptr++` (b) `ival++ && ival`
- (c) `vec[ival++] <= vec[ival]`

4.6. Операторы доступа к членам

Операторы точка (`.`) (dot operator) (см. раздел 1.5.2, стр. 51) и стрелка (`->`) (arrow operator) (см. раздел 3.4.1, стр. 158) обеспечивают доступ к члену. Оператор точка выбирает член из объекта типа класса; оператор стрелка определен так, что код `ptr->mem` эквивалентен коду `(*ptr).mem`.

```
string s1 = "a string", *p = &s1;
auto n = s1.size(); // вызов функции-члена size() строки s1
n = (*p).size();    // вызов функции-члена size() объекта, на который
                    // указывает указатель p
n = p->size();       // эквивалент (*p).size()
```

Поскольку приоритет обращения к значению ниже, чем оператора точка, часть обращения к значению следует заключить в скобки. Если пропустить круглые скобки, этот код поведет себя совсем по-иному:

```
// вызов функции-члена size() объекта, на который указывает указатель p
// затем обращение к значению результата!
*p.size(); // ошибка: p - указатель, он не имеет функции-члена size()
```

Этот код пытается вызвать функцию-член `size()` объекта `p`. Однако `p` — это указатель, у которого нет никаких членов; этот код не будет откомпилирован.

Оператор стрелка получает операнд в виде указателя и возвращает l-значение. Оператор точка возвращает l-значение, если объект, член которого выбирается, является l-значением; в противном случае результат — r-значение.

Упражнения раздела 4.6

Упражнение 4.20. С учетом того, что `iter` имеет тип `vector<string>::iterator`, укажите, какие из следующих выражений допустимы, если таковые имеются. Объясните поведение допустимых выражений, и почему ошибочные не допустимы?

- (a) `*iter++`; (b) `(*iter)++`; (c) `*iter.empty()`
 (d) `iter->empty()`; (e) `++*iter`; (f) `iter++->empty()`;

4.7. Условный оператор

Условный оператор (оператор `?:`) (conditional operator) позволяет внедрять простые конструкции `if...else` непосредственно в выражение. Условный оператор имеет следующий синтаксис:

условие ? *выражение1* : *выражение2*;

где *условие* — это выражение, используемое в качестве условия, а *выражение1* и *выражение2* — это выражения одинаковых типов (или типов, допускающих преобразование в общий тип). Эти выражения выполняются в зависимости от условия. Если условие истинно, то выполняется *выражение1*; в противном случае выполняется *выражение2*. В качестве примера использования условного оператора рассмотрим код, определяющий, является ли оценка (`grade`) проходной (`pass`) или нет (`fail`):

```
string finalgrade = (grade < 60) ? "fail" : "pass";
```

Условие проверяет, не меньше ли оценка 60. Если это так, то результат выражения `"fail"`; в противном случае — результат `"pass"`. Подобно операторам логического AND и OR (`&&` и `||`), условный оператор гарантирует, что выполнено будет только одно из выражений, *выражение1* или *выражение2*.

Результат условного оператора — l-значение, если оба выражения l-значения или если они допускают преобразование в общий тип l-значения. В противном случае результат — r-значение.

Вложенные условные операторы

Один условный оператор можно вложить в другой. Таким образом, условный оператор применяются как *условие* или как один или оба *выражения* другого условного оператора. В качестве примера используем пару вложенных условных операторов для трехступенчатой проверки оценки, чтобы выяснить, является ли она выше проходной, просто проходной или непроходной.

```
finalgrade = (grade > 90) ? "high pass"
              : (grade < 60) ? "fail" : "pass";
```

Первое условие проверяет, не выше ли оценка 90. Если это так, то выполняется выражение после `?`, возвращающее литерал `"high pass"`. Если условие ложно, выполняется ветвь `:`, которая сама является другим условным выражением. Это условное выражение проверяет, не меньше ли оценка 60. Если это так, то обрабатывается ветвь `?`, возвращающая литерал `"fail"`. В противном случае ветвь `:` возвращает литерал `"pass"`.

Условный оператор имеет правосторонний порядок, т.е. его операнды группируются (как обычно) справа налево. Порядок объясняет тот факт, что правое условное выражение, сравнивающее `grade` со значением 60, образует ветвь `:` левого условного выражения.



Вложенные условные выражения быстро становятся нечитабельными, поэтому нежелательно создавать больше двух или трех вложений.

Применение условного оператора в выражении вывода

Условный оператор имеет довольно низкий приоритет. Когда условный оператор используется в большом выражении, его, как правило, следует заключать в круглые скобки. Например, условный оператор нередко используют для отображения одного из значений в зависимости от результата проверки условия. Отсутствие круглых скобок вокруг условного оператора в выражении вывода может привести к неожиданным результатам:

```
cout << ((grade < 60) ? "fail" : "pass"); // выводит pass или fail
cout << (grade < 60) ? "fail" : "pass";   // выводит 1 или 0!
cout << grade < 60 ? "fail" : "pass";     // ошибка: сравнивает cout с 60
```

Второе выражение использует сравнение `grade` и 60 как операнд оператора `<<`. В зависимости от истинности или ложности выражения `grade < 60` выводится значение 1 или 0. Оператор `<<` возвращает объект `cout`, который

и проверяется в условии условного оператора. Таким образом, второе выражение эквивалентно следующему:

```
cout << (grade < 60);    // выводит 1 или 0
cout ? "fail" : "pass";  // проверяет cout, а затем возвращает один из
                        // этих двух литералов в зависимости от
                        // истинности объекта cout
```

Последнее выражение ошибочно, поскольку оно эквивалентно следующему:

```
cout << grade;           // приоритет оператора ниже, чем у
                        // сдвига, поэтому сначала выводится оценка,
cout < 60 ? "fail" : "pass"; // затем cout сравнивается с 60!
```

Упражнения раздела 4.7

Упражнение 4.21. Напишите программу, использующую условный оператор для поиска в векторе `vector<int>` элементов с нечетным значением и их удвоения.

Упражнение 4.22. Дополните программу, присваивающую переменной значение оценки (высокая, проходная, не проходная), еще одной оценки, минимально проходной, от 60 до 75 включительно. Напишите две версии: одна использует только условные операторы; вторая использует один или несколько операторов `if`. Как по вашему, какую версию проще понять и почему?

Упражнение 4.23. Следующее выражение не компилируется из-за приоритета операторов. Используя таблицу из раздела 4.12 (стр. 214), объясните причину проблемы. Как ее исправить?

```
string s = "word";
string pl = s + s[s.size() - 1] == 's' ? "" : "s" ;
```

Упражнение 4.24. Программа, различавшая проходную и непроходную оценку, зависела от того факта, что условный оператор имеет правосторонний порядок. Опишите, как обрабатывался бы этот оператор, имей он левосторонний порядок.

4.8. Побитовые операторы

Побитовые операторы (bitwise operator) получают операнды целочисленного типа, которые они используют как коллекции битов. Эти операторы позволяют проверять и устанавливать отдельные биты. Как будет описано в разделе 17.2 (стр. 906), эти операторы можно также использовать для библиотечного типа `bitset`, представляющего коллекцию битов изменяемого размера.

Как обычно, если операнд — “малое целое число”, его значение сначала преобразуется (раздел 4.11 стр. 219) в больший целочисленный тип. Операнды могут быть знаковыми или беззнаковыми.

Таблица 4.3. Побитовые операторы (левосторонний порядок)

Оператор	Действие	Применение
~	Побитовое NOT	<i>~ выражение</i>
<<	Сдвиг влево	<i>выражение1 << выражение2</i>
>>	Сдвиг вправо	<i>выражение1 >> выражение2</i>
&	Побитовое AND	<i>выражение1 & выражение2</i>
^	Побитовое XOR	<i>выражение1 ^ выражение2</i>
	Побитовое OR	<i>выражение1 выражение2</i>

Если операнд знаковый и имеет отрицательное значение, то способ обработки “знакового разряда” большинства битовых операций зависит от конкретной машины. Кроме того, результат сдвига влево, изменяющего знаковый разряд, непредсказуем.



Поскольку нет никаких гарантий однозначного выполнения побитовых операторов со знаковыми переменными на разных машинах, настоятельно рекомендуется использовать в них только беззнаковые целочисленные значения.

Побитовые операторы сдвига

Мы уже использовали перегруженные версии операторов >> и <<, которые библиотека IO определяет для ввода и вывода. Однако первоначальное значение этих операторов — побитовый сдвиг операндов. Они возвращают значение, являющееся копией (возможно преобразованной) левого операнда, биты которого сдвинуты. Правый операнд не должен быть отрицательным, и его значение должно быть меньше количества битов результата. В противном случае операция имеет неопределенный результат. Биты сдвигаются влево (<<) или право (>>), при этом вышедшие за пределы биты отбрасываются.

Оператор сдвига влево (<<) (left-shift operator) добавляет нулевые биты справа. Поведение *оператора сдвига вправо (>>) (right-shift operator)* зависит от типа левого операнда: если он беззнаковый, то оператор добавляет слева нулевые биты; если он знаковый, то результат зависит от конкретной реализации: слева вставляются либо копии знакового разряда, либо нули.

В этих примерах подразумевается, что младший бит расположен справа, тип `char` содержит 8 битов, а тип `int` — 32 бита

// 0233 - восьмеричный литерал (см. раздел 2.1.3, стр. 70)

unsigned char bits = 0233;

1	0	0	1	1	0	1	1
---	---	---	---	---	---	---	---

bits << 8 // bits преобразуется в int и сдвигается влево на 8 битов

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

bits << 31 // сдвиг влево на 31 бит отбрасывает крайние левые биты

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

bits >> 3 // сдвиг вправо на 3 бита отбрасывает 3 крайних правых бита

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Побитовый оператор NOT

Побитовый оператор NOT (~) (bitwise NOT operator) создает новое значение с инвертированными битами своего операнда. Каждый бит, содержащий 1, превращается в 0; каждый бит, содержащий 0, — в 1.

unsigned char bits = 0227;

1	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---

~bits

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	0	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Здесь операнд типа `char` сначала преобразуется в тип `int`. Это оставляет значение неизменным, но добавляет нулевые биты в позиции старших разрядов. Таким образом, преобразование в тип `int` добавляет 24 бита старших разрядов, заполненных нулями. Биты преобразованного значения инвертируются.

Побитовые операторы AND, OR и XOR

Побитовые операторы AND (&), OR (|) и XOR (^) создают новые значения с битовым шаблоном, состоящим из двух их операндов.

unsigned char b1 = 0145;

0	1	1	0	0	1	0	1
---	---	---	---	---	---	---	---

unsigned char b2 = 0257;

1	0	1	0	1	1	1	1
---	---	---	---	---	---	---	---

b1 & b2

Все 24 старших бита 0	0	0	1	0	0	1	0	1
-----------------------	---	---	---	---	---	---	---	---

b1 | b2

Все 24 старших бита 0	1	1	1	0	1	1	1	1
-----------------------	---	---	---	---	---	---	---	---

b1 ^ b2

Все 24 старших бита 0	1	1	0	0	1	0	1	0
-----------------------	---	---	---	---	---	---	---	---

Каждая битовая позиция результата побитового оператора AND (&) содержит 1, если оба операнда содержат 1 в этой позиции; в противном случае результат — 0. У побитового оператора OR (|) бит содержит 1, если один или оба операнда содержат 1; в противном случае результат — 0. Для побитового оператора XOR (^) бит содержит 1, если любой, но не оба операнда содержат 1; в противном случае результат — 0.



Побитовые и логические (см. раздел 4.3, стр. 197) операторы нередко путают. Например, путают побитовый оператор `&` с логическим `&&`, побитовый `|` с логическим `||` и побитовый `~` с логическим `!`.

Использование побитовых операторов

Рассмотрим пример использования побитовых операторов. Предположим, что есть класс с 30 учениками. Каждую неделю класс отвечает на контрольные вопросы с оценкой “сдано/не сдано”. Результаты всех контрольных записываются, по одному биту на ученика, чтобы представить успешную оценку или нет. Каждую контрольную можно представить в виде беззнакового целочисленного значения.

```
unsigned long quiz1 = 0; // это значение используется как коллекция битов
```

Переменная `quiz1` определена как `unsigned long`. Таким образом, на любой машине она будет содержать по крайней мере 32 бита. Переменная `quiz1` инициализируется явно, чтобы ее значение было определено изначально.

Учитель должен быть способен устанавливать и проверять отдельные биты. Например, должна быть возможность установить бит, соответствующий ученику номер 27, означающий, что этот ученик сдал контрольную. Чтобы указать, что ученик 27 прошел контрольную, создадим значение, у которого установлен только бит номер 27. Если затем применить побитовый оператор `OR` к этому значению и значению переменной `quiz1`, то все биты, кроме бита 27, останутся неизменными.

В данном примере счет битов переменной `quiz1` начинается с 0, соответствующего младшему биту, 1 соответствует следующему биту и т.д.

Чтобы получить значение, означающее, что ученик 27 сдал контрольную, используется оператор сдвига влево и целочисленный литерал 1 типа `unsigned long` (см. раздел 2.1.3, стр. 70).

```
1UL << 27 // создает значение только с одним установленным битом
           // в позиции 27
```

Первоначально переменная `1UL` имеет 1 в самом младшем бите и по крайней мере 31 нулевой бит. Она определена как `unsigned long`, поскольку тип `int` гарантированно имеет только 16 битов, а необходимо по крайней мере 27. Это выражение сдвигает 1 на 27 битовых позиций, вставляя в биты позади 0.

К этому значению и значению переменной `quiz1` применяется оператор `OR`. Поскольку необходимо изменить значение самой переменной `quiz1`, используем составной оператор присвоения (см. раздел 4.4, стр. 203):

```
quiz1 |= 1UL << 27; // указать, что ученик номер 27 сдал контрольную
```

Оператор `|=` выполняется аналогично оператору `+=`.

```
quiz1 = quiz1 | 1UL << 27; // эквивалент quiz1 |= 1UL << 27;
```

Предположим, что учитель пересмотрел контрольные и обнаружил, что ученик 27 фактически списал работу. Теперь учитель должен сбросить бит 27 в 0. На сей раз необходимо целое число, бит 27 которого сброшен, а все остальные установлены в 1. Применение побитового AND к этому значению и значению переменной `quiz1` позволяет сбросить только данный бит:

```
quiz1 &= ~(1UL << 27); // ученик номер 27 не прошел контрольную
```

Мы получаем значение со всеми установленными битами, кроме бита 27, инвертируя предыдущее значение. У него все биты были сброшены в 0, кроме бита 27, который был установлен в 1. Применение побитового NOT к этому значению сбросит бит 27, а все другие установит. Применение побитового AND к этому значению и значению переменной `quiz1` оставит неизменными все биты, кроме бита 27.

И наконец, можно узнать, как дела у ученика 27:

```
bool status = quiz1 & (1UL << 27); // как дела у ученика 27?
```

Здесь оператор AND применяется к значению с установленным битом 27 и значением переменной `quiz1`. Результат отличен от нуля (т.е. истинен), если бит 27 в значении переменной `quiz1` установлен; в противном случае он нулевой.



Операторы сдвига (они же ввода и вывода) имеют левосторонний порядок

Хотя многие программисты никогда не используют побитовые операторы непосредственно, почти все они используют их перегруженные версии в виде операторов ввода и вывода. Перегруженный оператор имеет тот же приоритет и порядок, что и его встроенная версия. Поэтому программисты должны иметь понятие о приоритете и порядке операторов сдвига, даже если они никогда не используют их встроенные версии.

Поскольку операторы сдвига имеют левосторонний порядок, выражение

```
cout << "hi" << " there" << endl;
```

выполняется так:

```
( (cout << "hi") << " there" ) << endl;
```

В этом операторе операнд `"hi"` группируется с первым символом `<<`. Его результат группируется со вторым, а его результат с третьим символом.

Приоритет операторов сдвига средний: ниже, чем у арифметических операторов, но выше, чем у операторов отношения, присвоения и условных опе-

раторов. Эти различия в уровнях приоритета свидетельствуют о том, что для правильной группировки операторов с более низким приоритетом следует использовать круглые скобки.

```
cout << 42 + 10;    // ok: приоритет + выше, поэтому выводится сумма
cout << (10 < 42); // ok: группировку определяют скобки; выводится 1
cout << 10 < 42;    // ошибка: попытка сравнить cout с 42!
```

Последний оператор `cout` интерпретируется так:

```
(cout << 10) < 42;
```

Он гласит: “записать 10 в поток `cout`, а затем сравнить результат (т.е. поток `cout`) со значением 42”.

Упражнения раздела 4.8

Упражнение 4.25. Каково значение выражения `~'q' << 6` на машине с 32-битовыми целыми числами и 8-битовыми символами, с учетом, что символ `'q'` имеет битовое представление 011110001?

Упражнение 4.26. Что будет, если в приведенном выше примере оценки учеников использовать для переменной `quiz1` тип `unsigned int`?

Упражнение 4.27. Каков результат каждого из этих выражений?

```
unsigned long u11 = 3, u12 = 7;
(a) u11 & u12          (b) u11 | u12
(c) u11 && u12          (d) u11 || u12
```

4.9. Оператор sizeof

Оператор `sizeof` возвращает размер в байтах результата выражения или указанного по имени типа. Оператор имеет правосторонний порядок. Результат оператора `sizeof` — это константное выражение (см. раздел 2.4.4, стр. 103) типа `size_t` (см. раздел 3.5.2, стр. 165). Оператор существует в двух формах.

```
sizeof(тип)
sizeof выражение
```

Во второй форме оператор `sizeof` возвращает размер типа, возвращаемого выражением. Оператор `sizeof` необычен тем, что он не выполняет свой операнд.

```
Sales_data data, *p;
sizeof(Sales_data); // размер, необходимый для хранения объекта
                    // типа Sales_item
sizeof data;        // размер типа данных, аналог sizeof(Sales_data)
sizeof p;           // размер указателя
sizeof *p;          // размер типа, на который указывает указатель p,
                    // т.е. sizeof(Sales_data)
```

```
sizeof data.revenue; // размер типа члена revenue класса Sales_data
sizeof Sales_data::revenue; // альтернативный способ получения
                          // размера revenue
```

Наиболее интересен пример `sizeof *p`. Во-первых, поскольку оператор `sizeof` имеет правосторонний порядок и тот же приоритет, что и оператор `*`, это выражение группируется справа налево. Таким образом, оно эквивалентно выражению `sizeof(*p)`. Во-вторых, поскольку оператор `sizeof` не выполняет свой операнд, не имеет значения, допустим ли указатель `p` (т.е. инициализирован ли он) (см. раздел 2.3.2, стр. 87). Обращения к значению недопустимого указателя оператор `sizeof` не осуществляет, и указатель фактически не используется, поэтому он безопасен. Ему и не нужно обращаться к значению указателя, чтобы выяснить, какой тип он возвратит.

**C++
11** По новому стандарту для доступа к члену класса при получении его размера можно использовать оператор области видимости. Обычно к членам класса можно обратиться только через объект этого класса. Больше не обязательно предоставлять объект, так как оператор `sizeof` не обязан выбирать член класса, чтобы узнать его размер.

Результат применения оператора `sizeof` частично зависит от типа, к которому он применен.

- Если это тип `char` или выражения, результат которого имеет тип `char`, то это гарантированно будет 1.
- Если это ссылка, то возвращает размер типа объекта, на который она ссылается.
- Если это указатель, то возвращается размер, необходимый для хранения указателя.
- Если это обращение к значению указателя, то возвращается размер типа объекта, на который он указывает, вне зависимости от его допустимости.
- Если это массив, то возвращается размер всего массива. Это эквивалентно получению размера элемента массива и его умножению на количество элементов. Обратите внимание, что оператор `sizeof` не преобразует массив в указатель.
- Если это строка или вектор, то возвращается размер только фиксированной части этих типов; но не размер, используемый элементами объекта.

Поскольку оператор `sizeof` возвращает размер всего массива, разделив размер массива на размер элемента, можно определить количество элементов в массиве:

```
// sizeof(ia)/sizeof(*ia) возвращает количество элементов в ia
constexpr size_t sz = sizeof(ia)/sizeof(*ia);
int arr2[sz]; // ok: sizeof возвращает константное выражение
              // (р. 2.4.4 стр. 103)
```

Так как оператор `sizeof` возвращает константное выражение, его результат можно использовать в выражении для определения размерности массива.

Упражнения раздела 4.9

Упражнение 4.28. Напишите программу для вывода размера каждого из встроенных типов.

Упражнение 4.29. Предскажите вывод следующего кода и объясните свое рассуждение. Напишите и выполните соответствующую программу. Совпадает ли вывод с ожиданиями? Если нет, то объясните почему.

```
int x[10];      int *p = x;
cout << sizeof(x)/sizeof(*x) << endl;
cout << sizeof(p)/sizeof(*p) << endl;
```

Упражнение 4.30. Используя таблицу из раздела 4.12 (стр. 214), расставьте скобки в следующих выражениях так, чтобы продемонстрировать порядок его обработки:

- | | |
|----------------------------------|--------------------------------------|
| (a) <code>sizeof x + y</code> | (b) <code>sizeof p->mem[i]</code> |
| (c) <code>sizeof a < b</code> | (d) <code>sizeof f()</code> |

4.10. Оператор запятая

Оператор запятая (,) (comma operator) получает два операнда, обрабатываемых слева направо. Подобно операторам логического AND и OR, а также условному оператору, оператор запятая гарантирует порядок обработки своих операндов.

Левое выражение обрабатывается, а его результат отбрасывается. Результат выражения запятая — это значение правого выражения. Результат является l-значением, если правый операнд — l-значение.

Оператор запятая нередко используется в цикле `for`:

```
vector<int>::size_type cnt = ivec.size();
// присвоить значения элементам size...l вектора ivec
for(vector<int>::size_type ix = 0;
    ix != ivec.size(); ++ix, --cnt)
    ivec[ix] = cnt;
```

Здесь выражения в заголовке цикла `for` увеличивают значение итератора `ix` и уменьшают значение целочисленной переменной `cnt`. Значения итератора `ix` и переменной `cnt` изменяются при каждой итерации цикла. Пока проверка итератора `ix` проходит успешно, следующему элементу присваивается текущее значение переменной `cnt`.

Упражнения раздела 4.10

Упражнение 4.31. Программа этого раздела использовала префиксные операторы инкремента и декремента. Объясните, почему были использованы префиксные, а не постфиксные версии? Что следует изменить для использования постфиксных версий? Перепишите программу с использованием постфиксных операторов.

Упражнение 4.32. Объясните следующий цикл:

```
constexpr int size = 5;
int ia[size] = {1,2,3,4,5};
for (int *ptr = ia, ix = 0;
     ix != size && ptr != ia+size;
     ++ix, ++ptr) { /* ... */ }
```

Упражнение 4.33. Используя таблицу раздела 4.12 (стр. 214), объясните, что делает следующее выражение:

```
someValue ? ++x, ++y : --x, --y
```



4.11. Преобразование типов

В языке C++ некоторые типы взаимосвязаны. Когда два типа взаимосвязаны, объект или значение одного типа можно использовать там, где ожидается операнд связанного типа. Два типа считаются связанными, если между ними возможно *преобразование* (conversion).

Для примера рассмотрим следующее выражение, инициализирующее переменную `ival` значением 6:

```
int ival = 3.541 + 3; // компилятор может предупредить о потере точности
```

Операндами сложения являются значения двух разных типов: 3.541 имеет тип `double` а 3 — `int`. Вместо попытки суммирования двух значений разных типов язык C++ определяет набор преобразований, позволяющих преобразовать операнды в общий тип. Эти преобразования выполняются автоматически без вмешательства программиста, а иногда и без его ведома. Поэтому они и называются *неявным преобразованием* (implicit conversion).

Неявные преобразования между арифметическими типами определены так, чтобы по возможности сохранять точность. Как правило, если у выражения есть и целочисленное значение, и значение с плавающей запятой, целое число преобразуется в число с плавающей точкой. В данном случае значение 3 преобразуется в тип `double`, осуществляется сложение чисел с плавающей запятой, и возвращается результат типа `double`.

Затем происходит инициализация. При инициализации доминирует тип инициализируемого объекта. Поэтому инициализатор преобразуется в его тип. В данном случае результат сложения типа `double` преобразуется в тип

`int` и используется для инициализации переменной `ival`. Преобразование типа `double` в тип `int` усекает значение типа `double`, отбрасывая десятичную часть. В данном случае выражение присваивает переменной `ival` значение 6.

Когда происходят неявные преобразования

Компилятор автоматически преобразует операнды при следующих обстоятельствах.

- В большинстве выражений значения целочисленных типов, меньших, чем `int`, сначала преобразуются в соответствующий больший целочисленный тип.
- В условиях не логические выражения преобразуются в тип `bool`.
- При инициализации инициализатор преобразуется в тип переменной; при присвоении правый операнд преобразуется в тип левого.
- В арифметических выражениях и выражениях отношения с операндами смешанных типов происходит преобразование в общий тип.
- Преобразования происходят также при вызове функций, как будет продемонстрировано в главе 6.



4.11.1. Арифметические преобразования

Арифметические преобразования (arithmetic conversion), впервые представленные в разделе 2.1.2 (стр. 66), преобразуют один арифметический тип в другой. Иерархию преобразований типов определяют правила, согласно которым операнды операторов преобразуются в самый большой общий тип. Например, если один операнд имеет тип `long double`, то второй операнд преобразуется тоже в тип `long double` независимо от своего типа. Короче говоря, в выражениях, где используются целочисленные значения и значения с плавающей точкой, целочисленное значение преобразуется в соответствующий тип с плавающей точкой.

Целочисленные преобразования

Целочисленное преобразование (integral promotion) преобразовывает значения малых целочисленных типов в большие. Типы `bool`, `char`, `signed char`, `unsigned char`, `short` и `unsigned short` преобразуются в `int`, если значение соответствует ему, а в противном случае оно преобразуется в тип `unsigned int`. Как уже неоднократно упоминалось, значение `false` типа `bool` преобразуется в 0, а `true` в 1.

Большие символьные типы (`wchar_t`, `char16_t` и `char32_t`) преобразуются в наименьший целочисленный тип `int`, `unsigned int`, `long`, `unsigned long`, `long long` или `unsigned long long`, которому соответствуют все возможные значения этого символьного типа.

Операнды беззнакового типа

Если операнды оператора имеют разные типы, они обычно преобразуются в общий тип. Если любой из операндов имеет беззнаковый тип, то тип, в который преобразуются операнды, зависит от относительных размеров целочисленных типов на машине.

Как обычно, сначала осуществляются целочисленные преобразования. Если полученные в результате типы совпадают, то никаких дальнейших преобразований не нужно. Если оба (возможно преобразованных) операнда имеют одинаковый знак, то операнд с меньшим типом преобразуется в больший тип.

При разных знаках, если тип беззнакового операнда больший, чем у знакового операнда, знаковый операнд преобразуется в беззнаковый. Например, при операторах типа `unsigned int` и `int`, `int` преобразуется в `unsigned int`. Следует заметить, что если значение типа `int` отрицательное, результат преобразуется так, как описано в разделе 2.1.2 (стр. 66).

Остается случай, когда тип знакового операнда больше, чем беззнакового. В данном случае результат зависит от конкретной машины. Если все значения беззнакового типа соответствуют большему типу, то операнд беззнакового типа преобразуется в знаковый. Если значения не соответствуют, то знаковый операнд преобразуется в беззнаковый. Например, если операнды имеют типы `long` и `unsigned int` и размер у них одинаковый, операнд типа `long` будет преобразован в `unsigned int`. Если тип `long` окажется больше, то `unsigned int` будет преобразован в `long`.

Концепция арифметических преобразований

Арифметические преобразования проще всего изучить на примерах.

```
bool flag; char cval;
short sval; unsigned short usval;
int ival; unsigned int uival;
long lval; unsigned long ulval;
float fval; double dval;
3.14159L + 'a'; // 'a' преобразуется в int, а затем int в long double
dval + ival;    // ival преобразуется в double
dval + fval;    // fval преобразуется в double
ival = dval;    // dval преобразуется в int (с усечением)
flag = dval;    // если dval - 0, flag - false, в противном случае - true
cval + fval;    // cval преобразуется в int, затем int во float
sval + cval;    // sval и cval преобразуется в int
cval + lval;    // cval преобразуется в long
ival + ulval;   // ival преобразуется в unsigned long
usval + ival;   // преобразование зависит от соотношения
                // размеров типов unsigned short и int
uival + lval;   // преобразование зависит от соотношения
                // размеров типов unsigned int и long
```

В первом выражении суммы символьная константа 'a' имеет тип `char`, являющийся числовым (см. раздел 2.1.1, стр. 62). Какое именно это значение, зависит от используемого машиной набора символов. На машине авторов, где установлен набор символов ASCII, символу 'a' соответствует число 97. При добавлении символа 'a' к значению типа `long double` значение типа `char` преобразуется в тип `int`, а затем в тип `long double`. Это преобразованное значение добавляется к литералу. Интересны также два последних случая, где происходит преобразование беззнаковых значений. Тип результата этих выражений зависит от конкретной машины.

Упражнения раздела 4.11.1

Упражнение 4.34. С учетом определений переменных данного раздела объясните, какие преобразования имеют место в следующих выражениях:

(a) `if (fval)` (b) `dval = fval + ival;` (c) `dval + ival * cval;`

Помните, что возможно придется учитывать порядок операторов.

Упражнение 4.35. С учетом определений

```
char cval;      int ival;      unsigned int ui;
float fval;     double dval;
```

укажите неявные преобразования типов, если таковые вообще имеются.

(a) `cval = 'a' + 3;` (b) `fval = ui - ival * 1.0;`
 (c) `dval = ui * fval;` (d) `cval = ival + fval + dval;`



4.11.2. Другие неявные преобразования

Кроме арифметических, существует еще несколько видов неявных преобразований, включая следующие.

Преобразование массива в указатель. В большинстве выражений, когда используется массив, он автоматически преобразуется в указатель на свой первый элемент.

```
int ia[10];      // массив из десяти целых чисел
int* ip = ia;    // ia преобразуется в указатель на первый элемент
```

Это преобразование не происходит при использовании массива в выражении `decltype` или в качестве операнда операторов обращения к адресу (`&`), `sizeof` или `typeid` (который рассматривается в разделе 19.2.2 (стр. 1032)). Преобразование не происходит также при инициализации ссылки на массив (см. раздел 3.5.1, стр. 162). Подобное преобразование указателя происходит при использовании в выражении типа функции, как будет описано в разделе 6.7 (стр. 322).

Преобразование указателя. Существует несколько других преобразований указателя: постоянное целочисленное значение 0 и литерал `nullptr` могут

быть преобразованы в указатель на любой тип; указатель на любой неконстантный тип может быть преобразован в `void*`, а указатель на любой тип может быть преобразован в `const void*`. Как будет продемонстрировано в разделе 15.2.2 (стр. 754), существуют дополнительные преобразования указателя, относящиеся к типам, связанным наследованием.

Преобразование в тип `bool`. Существует автоматическое преобразование арифметических типов и типов указателя в тип `bool`. Если указатель или арифметическое значение — нуль, преобразование возвращает значение `false`; любое другое значение возвращает `true`:

```
char *cp = get_string();
if (cp) /* ... */ // true, если cp не нулевой указатель
while (*cp) /* ... */ // true, если *cp не нулевой символ
```

Преобразование в константу. Указатель на неконстантный тип можно преобразовать в указатель на соответствующий константный тип, то же относится и к ссылкам. Таким образом, если `T` — тип, то указатель или ссылку на тип `T` можно преобразовать в указатель или ссылку на `const T` (см. разделы 2.4.1, стр. 97 и 2.4.2, стр. 99 соответственно).

```
int i;
const int &j = i; // преобразовать в ссылку на const int
const int *p = &i; // преобразовать неконстантный адрес в константный
int &r = j, *q = p; // ошибка: преобразование константы в не константу
// недопустимо
```

Обратное преобразование (устранение спецификатора `const` нижнего уровня) невозможно.

Преобразование, определенное типами класса. Тип класса может сам определять преобразования, которые компилятор применит автоматически. Компилятор применяет только одно преобразование типа класса за раз. В разделе 7.5.4 (стр. 381) приведен пример, когда необходимо несколько преобразований, и он не работает.

В программах ранее уже использовались преобразования типов класса, когда символьная строка в стиле `C` использовалась там, где ожидался библиотечный тип `string` (см. раздел 3.5.5, стр. 175), а также при чтении из потока `istream` в условии.

```
string s, t = "a value"; // символьный строковый литерал преобразован
                        // в тип string
while (cin >> s) // условие while преобразует cin в bool
```

Условие `(cin >> s)` читает поток `cin` и возвращает его же как результат. Условия ожидают значение типа `bool`, но оно проверяет значение типа `istream`. Библиотека `IO` определяет преобразование из типа `istream` в `bool`. Это преобразование используется автоматически, чтобы преобразовать поток `cin` в тип `bool`. Результирующее значение типа `bool` зависит от состояния по-

тока. Если последнее чтение успешно, то преобразование возвращает значение `true`. Если последняя попытка потерпела неудачу, то преобразование возвращает значение `false`.

4.11.3. Явные преобразования

Иногда необходимо явно преобразовать объект в другой тип. Например, в следующем коде может понадобиться использование деления с плавающей точкой:

```
int i, j;  
double slope = i/j;
```

Для этого необходим способ явного преобразования переменных `i` и/или `j` в тип `double`. Для явного преобразования используется *приведение* (`cast`) типов.



ВНИМАНИЕ

Хотя приведение время от времени необходимо, оно довольно опасно.

Именованные операторы приведения

Именованный оператор приведения имеет следующую форму:

```
имя_приведения<тип> (выражение) ;
```

где *тип* — это результирующий тип преобразования, а *выражение* — приводимое значение. Если *тип* — ссылка, то результат l-значение. *Имя_приведения* может быть одним из следующих: `static_cast`, `dynamic_cast`, `const_cast` и `reinterpret_cast`. Приведение `dynamic_cast`, обеспечивающее идентификацию типов времени выполнения, рассматривается в разделе 19.2 (стр. 1029). *Имя_приведения* определяет, какое преобразование осуществляется.

Оператор `static_cast`

Любое стандартное преобразование типов, кроме действующего спецификатор `const` нижнего уровня, можно затребовать, используя оператор `static_cast`. Например, приведя тип одного из операндов к типу `double`, можно заставить выражение использовать деление с плавающей точкой:

```
// приведение для вынужденного деления с плавающей точкой  
double slope = static_cast<double>(j) / i;
```

Оператор `static_cast` зачастую полезен при присвоении значения большего арифметического типа переменной меньшего. Приведение сообщает и читателю программы, и компилятору, что мы знаем и не беспокоимся о возможной потере точности. При присвоении большего арифметического типа меньшему компиляторы зачастую выдают предупреждение. При явном приведении предупреждающее сообщение не выдается.

Оператор `static_cast` полезен также при выполнении преобразований, которые компилятор не выполняет автоматически. Например, его можно использовать для получения значения указателя, сохраняемого в указателе `void*` (см. раздел 2.3.2, стр. 92):

```
void* p = &d; // ok: адрес любого неконстантного объекта может храниться
              // в указателе void*
// ok: преобразование void* назад в исходный тип указателя
double *dp = static_cast<double*>(p);
```

После сохранения адреса в указателе типа `void*` можно впоследствии использовать оператор `static_cast` и привести указатель к его исходному типу, что позволит сохранить значение указателя. Таким образом, результат приведения будет равен первоначальному значению адреса. Однако следует быть абсолютно уверенным в том, что тип, к которому приводится указатель, является фактическим типом этого указателя; при несоответствии типов результат непредсказуем.

Оператор `const_cast`

Оператор `const_cast` изменяет только спецификатор `const` нижнего уровня своего операнда (см. раздел 2.4.3, стр. 101):

```
const char *pc;
char *p = const_cast<char*>(pc); // ok: однако запись при помощи p
                                // указателя непредсказуема
```

Принято говорить, что приведение, преобразующее константный объект в неконстантный, “сбрасывает `const`”. При сбросе константности объекта компилятор больше не будет препятствовать записи в этот объект. Если объект первоначально не был константным, использование приведения для доступа на запись вполне допустимо. Но применение оператора `const_cast` для записи в первоначально константный объект непредсказуемо.

Только оператор `const_cast` позволяет изменить константность выражения. Попытка изменить константность выражения при помощи любого другого именованного оператора приведения закончится ошибкой компиляции. Аналогично нельзя использовать оператор `const_cast` для изменения типа выражения:

```
const char *cp;
// ошибка: static_cast не может сбросить const
char *q = static_cast<char*>(cp);
static_cast<string>(cp); // ok: преобразует строковый литерал в строку
const_cast<string>(cp); // ошибка: const_cast изменяет только
                        // константность
```

Оператор `const_cast` особенно полезен в контексте перегруженных функций, рассматриваемых в разделе 6.4 (стр. 305).

Оператор `reinterpret_cast`

Оператор `reinterpret_cast` осуществляет низкоуровневую интерпретацию битовой схемы своих операндов. Рассмотрим, например, следующее приведение:

```
int *ip;  
char *pc = reinterpret_cast<char*>(ip);
```

Никогда не следует забывать, что фактическим объектом, на который указывает указатель `pc`, является целое число, а не символ. Любое использование указателя `pc`, подразумевающее, что это обычный символьный указатель, вероятно, потерпит неудачу во время выполнения. Например, следующий код, вероятней всего, приведет к непредвиденному поведению во время выполнения:

```
string str(pc);
```

Использование указателя `pc` для инициализации объекта типа `string` — хороший пример небезопасности оператора `reinterpret_cast`. Проблема в том, что при изменении типа компилятор не выдаст никаких предупреждений или сообщений об ошибке. При инициализации указателя `pc` адресом типа `int` компилятор не выдаст ни предупреждения, ни сообщения об ошибке, поскольку явно указано, что это и нужно. Однако любое последующее применение указателя `pc` подразумевает, что он содержит адрес значения типа `char*`. Компилятор не способен выяснить, что фактически это указатель на тип `int`. Таким образом, инициализация строки `str` при помощи указателя `pc` вполне правомерна, хотя в данном случае абсолютно бессмысленна, если не хуже! Отследить причину такой проблемы иногда чрезвычайно трудно, особенно если приведение указателя `ip` к `pc` происходит в одном файле, а использование указателя `pc` для инициализации объекта класса `string` — в другом.



ВНИМАНИЕ

Оператор `reinterpret_cast` жестко зависит от конкретной машины. Чтобы безопасно использовать оператор `reinterpret_cast`, следует хорошо понимать, как именно реализованы используемые типы, а также то, как компилятор осуществляет приведение.

Приведение типов в старом стиле

В ранних версиях языка C++ явное приведение имело одну из следующих двух форм:

```
тип (выражение); // форма записи приведения в стиле функции  
(тип) выражение; // форма записи приведения в стиле языка C
```

В зависимости от используемых типов, приведение старого стиля срабатывает аналогично операторам `const_cast`, `static_cast` или `reinterpret_cast`. В случаях, где используются операторы `static_cast` или `const_cast`, приве-

дение типов в старом стиле позволяет осуществить аналогичное преобразование, что и соответствующий именованный оператор приведения. Но если ни один из подходов не допустим, то приведение старого стиля срабатывает аналогично оператору `reinterpret_cast`. Например, используя форму записи старого стиля, можно получить тот же результат, что и с использованием `reinterpret_cast`.

```
char *pc = (char*) ip; // ip указатель на тип int
```

СОВЕТ. ИЗБЕГАЙТЕ ПРИВЕДЕНИЯ ТИПОВ

Приведение нарушает обычный порядок контроля соответствия типов (см. раздел 2.2, стр. 80), поэтому авторы настоятельно рекомендуют избегать приведения типов. Это особенно справедливо для оператора `reinterpret_cast`. Такие приведения всегда опасны. Операторы `const_cast` могут быть весьма полезны в контексте перегруженных функций, рассматриваемых в разделе 6.4 (стр. 305). Использование оператора `const_cast` зачастую свидетельствует о плохом проекте. Другие операторы приведения, `static_cast` и `dynamic_cast`, должны быть необходимы нечасто. При каждом применении приведения имеет смысл хорошо подумать, а нельзя ли получить тот же результат другим способом. Если приведение все же неизбежно, имеет смысл принять меры, позволяющие снизить вероятность возникновения ошибки, т.е. ограничить область видимости, в которой используется приведенное значение, а также хорошо документировать все подобные случаи.



ВНИМАНИЕ

Приведения старого стиля менее очевидны, чем именованные операторы приведения. Поскольку их легко упустить из виду, обнаружить ошибку становится еще трудней.

Упражнения раздела 4.11.3

Упражнение 4.36. С учетом того, что `i` имеет тип `int`, а `d` — `double`, напишите выражение `i *= d` так, чтобы осуществлялось целочисленное умножение, а не с плавающей запятой.

Упражнение 4.37. Перепишите каждое из следующих приведений старого стиля так, чтобы использовался именованный оператор приведения.

```
int i; double d; const string *ps; char *pc; void *pv;
```

- (a) `pv = (void*)ps;` (b) `i = int(*pc);`
 (c) `pv = &d;` (d) `pc = (char*) pv;`

Упражнение 4.38. Объясните следующее выражение:

```
double slope = static_cast<double>(j/i);
```

4.12. Таблица приоритетов операторов

Таблица 4.4. Приоритет операторов

Порядок и оператор	Действие	Применение	Стр.
$\Lambda ::$	Глобальная область видимости	$:: \text{имя}$	371
$\Lambda ::$	Область видимости класса	$\text{класс} :: \text{имя}$	131
$\Lambda ::$	Область видимости пространства имен	$\text{пространствоимен} :: \text{имя}$	124
$\Lambda .$	Обращение к члену класса	объект.член	51
$\Lambda \rightarrow$	Обращение к члену класса	$\text{pointer} \rightarrow \text{член}$	158
$\Lambda []$	Индексирование	$\text{выражение}[\text{выражение}]$	165
$\Lambda ()$	Вызов функции	$\text{имя}(\text{список_выражений})$	52
$\Lambda ()$	Конструкция type	$\text{тип}(\text{список_выражений})$	225
$\text{P}++$	Постфиксный инкремент	$l\text{-значение}++$	204
$\text{P}--$	Постфиксный декремент	$l\text{-значение}--$	204
P typeid	Идентификатор типа	$\text{typeid}(\text{тип})$	1032
P typeid	Идентификатор типа времени выполнения	$\text{typeid}(\text{выражение})$	1032
$\text{P Явное приведение}$	Преобразование типов	$\text{cast_имя} < \text{тип} > (\text{выражение})$	223
$\text{P}++$	Префиксный инкремент	$++l\text{-значение}$	204
$\text{P}--$	Префиксный декремент	$--l\text{-значение}$	204
$\text{P} \sim$	Побитовое NOT	$\sim \text{выражение}$	212
$\text{P} !$	Логическое NOT	$! \text{выражение}$	199
$\text{P} -$	Унарный минус	$- \text{выражение}$	195
$\text{P} +$	Унарный плюс	$+ \text{выражение}$	194
$\text{P} *$	Обращение к значению	$* \text{выражение}$	88
$\text{P} \&$	Обращение к адресу	$\&l\text{-значение}$	87
$\text{P} ()$	Преобразование типов	$(\text{тип}) \text{ выражение}$	215
P sizeof	Размер объекта	$\text{sizeof } \text{выражение}$	215
P sizeof	Размер типа	$\text{sizeof}(\text{тип})$	215
$\text{P sizeof} \dots$	Размер пакета параметров	$\text{sizeof} \dots (\text{имя})$	215
P new	Создание объекта	new тип	584
$\text{P new} []$	Создание массива	$\text{new тип}[\text{размер}]$	584
P delete	Освобождение объекта	delete выражение	587
$\text{P delete} []$	Освобождение массива	$\text{delete}[] \text{ выражение}$	587
P noexcept	Способность к передаче	$\text{noexcept}(\text{выражение})$	976

Окончание табл. 4.4

Порядок и оператор	Действие	Применение	Стр.
$\wedge \rightarrow *$	Указатель на член класса	<i>указатель- >*указатель_на_член</i>	1044
$\wedge . *$	Указатель на член класса	<i>объ- ект.*указатель_на_член</i>	1044
$\wedge *$	Умножение	<i>выражение * выражение</i>	196
$\wedge /$	Деление	<i>выражение / выражение</i>	196
$\wedge \%$	Деление по модулю (остаток)	<i>выражение % выражение</i>	196
$\wedge +$	Сумма	<i>выражение + выражение</i>	196
$\wedge -$	Разница	<i>выражение - выражение</i>	196
$\wedge <<$	Побитовый сдвиг влево	<i>выражение << выражение</i>	211
$\wedge >>$	Побитовый сдвиг вправо	<i>выражение >> выражение</i>	211
$\wedge <$	Меньше	<i>выражение < выражение</i>	197
$\wedge <=$	Меньше или равно	<i>выражение <= выражение</i>	197
$\wedge >$	Больше	<i>выражение > выражение</i>	197
$\wedge >=$	Больше или равно	<i>выражение >= выражение</i>	197
$\wedge ==$	Равенство	<i>выражение == выражение</i>	197
$\wedge !=$	Неравенство	<i>выражение != выражение</i>	197
$\wedge \&$	Побитовый AND	<i>выражение & выражение</i>	212
$\wedge ^$	Побитовый XOR	<i>выражение ^ выражение</i>	212
$\wedge $	Побитовый OR	<i>выражение выражение</i>	212
$\wedge \&\&$	Логический AND	<i>выражение &\& выражение</i>	198
$\wedge $	Логический OR	<i>выражение выражение</i>	198
$\Pi ? :$	Условный оператор	<i>выражение ? выражение : выражение</i>	208
$\Pi =$	Присвоение	<i>l-значение = выражение</i>	201
$\Pi *=, /=, \%=,$	Составные	<i>l-значение += выражение,</i>	203
$\Pi +=, -=,$	Операторы	и т.д.	203
$\Pi <<=, >>=,$	присвоения		203
$\Pi \&=, =, ^=$			203
$\Pi \text{ throw}$	Передача исключения	<i>throw выражение</i>	258
$\wedge ,$	Запятая	<i>выражение , выражение</i>	217

Резюме

Язык C++ предоставляет богатый набор операторов и определяет их назначение, когда они относятся к значениям встроенных типов. Кроме того, язык поддерживает перегрузку операторов, позволяющую самостоятельно определять назначение операторов для типов класса. Определение операторов для собственных типов рассматривается в главе 14.

Чтобы разобраться в составных выражениях (содержащих несколько операторов), необходимо выяснить приоритет и порядок обработки операндов. Каждый оператор имеет приоритет и порядок. Приоритет определяет группировку операторов в составном выражении, а порядок определяет группировку операторов с одинаковым уровнем приоритета.

Для большинства операторов порядок выполнения операндов не определен, компилятор выбирает сам, какой операнд обработать сначала — левый или правый. Зачастую порядок вычисления результатов операндов никак не влияет на результат выражения. Но если оба операнда обращаются к одному объекту, причем один из них *изменяет* объект, то порядок выполнения становится весьма важен, а связанные с ним серьезные ошибки обнаружить крайне сложно.

И наконец, компилятор зачастую сам преобразует тип операндов в другой связанный тип. Например, малые целочисленные типы преобразуются в больший целочисленный тип каждого выражения. Преобразования существуют и для встроенных типов, и для классов. Преобразования могут быть также осуществлены явно, при помощи приведения.

Термины

L-значение (l-value). Выражение, возвращающее объект или функцию. Неконстантное l-значение обозначает объект, который может быть левым операндом оператора присвоения.

R-значение (r-value). Выражение, возвращающее значение, но не ассоциированную с ним область, если таковое значение вообще имеется.

Арифметическое преобразование (arithmetic conversion). Преобразование одного арифметического типа в другой. В контексте парных арифметических операторов арифметические преобразования, как правило, сохраняют точность, преобразуя значения меньшего типа в значения большего (например, меньший целочисленный тип `char` или `short` преобразуется в `int`).

Выражение (expression). Самый низкий уровень вычислений в программе на языке C++. Как правило, выражения состоят из одного или нескольких операторов. Каждое выражение возвращает результат. Выражения могут использоваться в качестве операндов, что позволяет создавать составные выражения, которым для вычисления собственного результата нужны результаты других выражений, являющихся его операндами.

Вычисление по сокращенной схеме (short-circuit evaluation). Термин, описывающий способ выполнения операторов логического AND и OR. Если первого операнда этих операторов достаточно для определения общего результата, то остальные операнды не рассматриваются и не вычисляются.

Неявное преобразование (implicit conversion). Преобразование, которое осуществляется компилятором автоматически. Такое преобразование осуществляется в случае, когда оператор получает значение, тип которого отличается от необходимого. Компилятор автоматически преобразует операнд в необходимый тип, если соответствующее преобразование определено.

Операнд (operand). Значение, с которым работает выражение. У каждого оператора есть один или несколько операндов

Оператор --. Оператор декремента. Имеет две формы, префиксную и постфиксную. Префиксный оператор декремента возвращает l-значение. Он вычитает единицу из значения операнда и возвращает полученное значение. Постфиксный оператор декремента возвращает r-значение. Он вычитает единицу из значения операнда, но возвращает исходное, неизмененное значение. Примечание: итераторы имеют оператор --, даже если у них нет оператора -.

Оператор !. Оператор логического NOT. Возвращает инверсное значение своего операнда типа bool. Результат true, если операнд false, и наоборот.

Оператор &. Побитовый оператор AND. Создает новое целочисленное значение, в котором каждая битовая позиция имеет значение 1, если оба операнда в этой позиции имеют значение 1. В противном случае бит получает значение 0.

Оператор &&. Оператор логического AND. Возвращает значение true, если оба операнда истинны. Правый операнд обрабатывается, *только если* левый операнд истинен.

Оператор ,. Оператор запятая. Бинарный оператор, обрабатывающийся слева направо. Результатом оператора запятая является значение справа. Результат является l-значением, только если его операнд — l-значение.

Оператор ?:. Условный оператор. Сокращенная форма конструкции if...else следующего вида: *условие ? выражение1 : выражение2*; Если *условие* истинно (значение true) выполняется *выражение1*, в противном случае — *выражение2*. Тип выражений должен совпадать или допускать преобразование в общий тип. Выполняется только одно из выражений.

Оператор ^. Побитовый оператор XOR. Создает новое целочисленное значение, в котором каждая битовая позиция имеет значение 1, если любой (но не оба) из операндов содержит значение 1 в этой битовой позиции. В противном случае бит получает значение 0.

Оператор |. Побитовый оператор OR. Создает новое целочисленное значение, в котором каждая битовая позиция имеет значение 1, если любой из операндов содержит значение 1 в этой битовой позиции. В противном случае бит получает значение 0.

Оператор ||. Оператор логического OR. Возвращает значение true, если любой из операндов истинен. Правый операнд обрабатывается, *только если* левый операнд ложен.

Оператор ~. Побитовый оператор NOT. Инвертирует биты своего операнда.

Оператор ++. Оператор инкремента. Оператор инкремента имеет две формы, префиксную и постфиксную. Префиксный оператор инкремента возвращает l-значение. Он добавляет единицу к значению операнда и возвращает полученное значение. Пост-

фиксный оператор инкремента возвращает *i*-значение. Он добавляет единицу к значению операнда, но возвращает исходное, неизмененное значение. Примечание: итераторы имеют оператор ++, даже если у них нет оператора +.

Оператор <<. Оператор сдвига влево. Сдвигает биты левого операнда влево. Количество позиций, на которое осуществляется сдвиг, задает правый операнд. Правый операнд должен быть нулем или положительным значением, ни в коем случае не превосходящим количества битов в левом операнде. Левый операнд должен быть беззнаковым; если левый операнд будет иметь знаковый тип, то сдвиг бита знака приведет к непредсказуемому результату.

Оператор >>. Оператор сдвига вправо. Аналогичен оператору сдвига влево, за исключением направления перемещения битов. Правый операнд должен быть нулем или положительным значением, ни в коем случае не превосходящим количества битов в левом операнде. Левый операнд должен быть беззнаковым; если левый операнд будет иметь знаковый тип, то сдвиг бита знака приведет к непредсказуемому результату.

Оператор const_cast. Применяется при преобразовании объекта со спецификатором const нижнего уровня в соответствующий неконстантный тип, и наоборот.

Оператор dynamic_cast. Используется в комбинации с наследованием и идентификацией типов во время выполнения. См. раздел 19.2, стр. 1029.

Оператор reinterpret_cast. Интерпретирует содержимое операнда как другой тип. Очень опасен и жестко зависит от машины.

Оператор sizeof. Возвращает размер в байтах объекта, указанного по имени типа, или типа переданного выражения.

Оператор static_cast. Запрос на явное преобразование типов, которое компилятор осуществил бы неявно. Зачастую используется для переопределения неявного преобразования, которое в противном случае выполнил бы компилятор.

Оператор (operator). Символ, который определяет действие, выполняемое выражением. В языке определен целый набор операторов, которые применяются для значений встроенных типов. В языке определен также приоритет и порядок выполнения для каждого оператора, а также задано количество операндов для каждого из них. Операторы могут быть перегружены и применены к объектам классов.

Парный оператор (binary operator). Операторы, в которых используются два операнда.

Перегруженный оператор (overloaded operator). Версия оператора, определенного для использования с объектом класса. Определение перегруженных версий операторов рассматривается в главе 14.

Порядок (associativity). Определяет последовательность выполнения операторов одинакового приоритета. Операторы могут иметь правосторонний (справа налево) или левосторонний (слева направо) порядок выполнения.

Порядок вычисления (order of evaluation). Порядок, если он есть, определяет последовательность вычисления операндов оператора. В большинстве случаев компилятор C++ самостоятельно определяет порядок вычисления операндов. Однако, прежде чем выполнится сам оператор, всегда вычисляются его операнды. Только операторы &&, ||, ?: и , определяют порядок выполнения своих операндов.

Преобразование (conversion). Процесс, в ходе которого значение одного типа преобразуется в значение другого типа. Преобразования между встроенными типами заложены в самом языке. Для классов также возможны преобразования типов.

Преобразование (promotion). См. целочисленное преобразование.

Приведение (cast). Явное преобразование типов.

Приоритет (precedence). Определяет порядок выполнения операторов в выражении.

Операторы с более высоким приоритетом выполняются прежде операторов с более низким приоритетом.

Результат (result). Значение или объект, полученный при вычислении выражения.

Составное выражение (compound expression). Выражение, состоящее из нескольких операторов.

Унарный оператор (unary operator). Оператор, использующий один операнд.

Целочисленное преобразование (integral promotion). Подмножество стандартных преобразований, при которых меньший целочисленный тип приводится к ближайшему большему типу. Операнды меньших целочисленных типов (например, `short`, `char` и т.д.) преобразуются всегда, даже если такие преобразования, казалось бы, необязательны.